CAMPAIGN RESULTS

Remote Training Validation Campaign Results Report

Remote LAN Training Validation

Styled PDF edition generated from the canonical Markdown analysis report for improved readability, section hierarchy, and table presentation.

Overview

This report summarizes the first real LAN-backed training campaign prepared in:

- `doc/reports/campaign_plans/2026-04-03-17-54-21_remote_training_validation_campaign_plan_report.md`

  The campaign was designed to answer two questions at once:
- whether a stronger LAN workstation is actually useful for the higher-memory

  tree follow-up, especially the random-forest branch;
- whether the repository-owned remote launcher can execute and return a mixed

  campaign spanning both `scikit-learn` tree runs and GPU-backed Lightning feedforward runs. The real execution completed on the remote workstation with one modeling failure and one workflow-bookkeeping anomaly:
- completed runs: `4`
- failed runs: `1`
- execution report generated at: `2026-04-03T22:30:26`
- campaign execution window: `2026-04-03T20:38:07` to `2026-04-03T22:30:26`

  The campaign artifact root is:
- `output/training_campaigns/2026-04-03-20-38-07_remote_training_validation_campaign_2026_04_03_17_54_21/`

Objective And Outcome

This campaign was intentionally narrow. It was not a broad new model-family research wave. It was a practical validation pass that needed to:

- run a meaningful tree follow-up on the stronger LAN workstation;
- run at least one materially heavier neural configuration on remote GPU;
- validate source sync, remote execution, campaign bookkeeping, registry

  updates, and artifact synchronization back to the local repository. On the modeling side, the outcome is useful but not transformative:
- the deeper remote `hist_gradient_boosting` run became the campaign winner;
- the remote feedforward runs produced a new family-level best for the

  feedforward family;
- the random-forest branch still did not become competitive, and the more

  aggressive configuration failed again due to memory pressure. On the workflow side, the result is mixed but still successful enough to keep:
- the remote path executed real mixed-family training end to end;
- the remote clone, Conda environment, GPU path, queue runner, and registries

  all worked on the LAN workstation;

- the launcher exposed three implementation defects during the first real run: `GetRelativePath` compatibility, PowerShell binary-stream corruption during tar piping, and `CLIXML` progress serialization on remote PowerShell output;
- these defects were corrected locally during the validation pass;
- one campaign-manifest bookkeeping bug remains visible because the medium random-forest run completed, but the manifest serialized the wrong `run_instance_id` by one second.

Ranking Policy

The campaign ranking follows the serialized policy stored in `campaign_leaderboard.yaml`:

- primary metric: `test_mae`
- first tie breaker: `test_rmse`
- second tie breaker: `val_mae`
- third tie breaker: `trainable_parameter_count`

Campaign Ranking

Ranked Completed Runs

Rank	Config	Family	Runtime	Test MAE [deg]	Test RMSE [deg]	Val MAE [deg]
1	`te_hist_gbr_remote_deep`	`tree`	1.9 min	0.002920	0.003644	0.002749
2	`te_feedforward_high_compute_remote`	`feedforward`	10.4 min	0.003274	0.003873	0.003059
3	`te_feedforward_stride1_big_remote`	`feedforward`	29.9 min	0.003278	0.003671	0.003019
4	`te_random_forest_remote_medium`	`tree`	39.2 min	0.003865	0.004861	0.003808

Failed Run

Config	Family	Runtime	Outcome
`te_random_forest_remote_aggressive`	`tree`	30.8 min	Failed with `could not allocate 536870912 bytes`

Campaign Winner

The explicit campaign winner is

- `te_hist_gbr_remote_deep`

  Its result was:
- validation MAE: `0.002749 deg`
- validation RMSE: `0.003664 deg`
- test MAE: `0.002920 deg`
- test RMSE: `0.003644 deg`

  This run won the campaign because it remained the lowest-test-MAE entry among the completed remote-validation candidates.

Interpretation By Model Branch

1. HistGradientBoosting Remains The Strongest Practical Family

The stronger remote `hist_gradient_boosting` follow-up was fast and stable:

- runtime: `1.9 min`
- validation MAE: `0.002749 deg`
- test MAE: `0.002920 deg`

  Interpretation:
- the family remains the strongest practical branch in this repository;
- the remote deeper run did not beat the existing global program leader

  `te_hist_gbr_tabular` at `test_mae = 0.002885 deg` ;
- the gap is small, but the result still says the earlier tree leader was

  already close to saturation. This means the stronger remote workstation did not reveal an obviously better tree regime than the current global best. It mostly confirmed the existing tree-family anchor.

2. Random Forest Still Looks Unattractive Even On The Stronger Machine

The two random-forest runs produced the clearest practical answer from the LAN test.

The medium remote retry completed

- validation MAE: `0.003808 deg`
- test MAE: `0.003865 deg`
- runtime: `39.2 min`

The aggressive retry failed:

- failure: `could not allocate 536870912 bytes`
- runtime before failure: `30.8 min`

Interpretation:

- the stronger workstation was enough to complete a less-constrained random-forest run;
- that completed run is still slightly worse than the earlier conservative recovery random forest at `test_mae = 0.003833 deg`;
- the more aggressive configuration still hits a memory wall even on the LAN workstation. So the remote test answers the open random-forest question fairly clearly:
- extra workstation strength helps operationally;
- it does not currently justify keeping random forest as a mainline family for best TE prediction;
- the aggressive branch still scales poorly in memory and artifact size.

3. The Remote GPU Path Produced A Better Feedforward Family Best

Both remote neural runs completed successfully and validated the Lightning path on the remote GPU.

`te_feedforward_high_compute_remote`

- validation MAE: `0.003059 deg`
- test MAE: `0.003274 deg`
- test RMSE: `0.003873 deg`
- runtime: `10.4 min`

`te_feedforward_stride1_big_remote`:

- validation MAE: `0.003019 deg`
- test MAE: `0.003278 deg`
- test RMSE: `0.003671 deg`
- runtime: `29.9 min`

Interpretation:

- the remote GPU path is operationally validated: checkpoints, metrics, reports, and registries were produced correctly for both feedforward runs;
- `te_feedforward_high_compute_remote` became the new feedforward family best, improving over the earlier family-best baseline `te_feedforward_stride5_long_large_batch` at `test_mae = 0.003301 deg`;
- the stride-1 dense run improves validation MAE and test RMSE relative to the high-compute run, but it does not beat it on the primary ranking metric `test_mae`, so the registry winner remains the high-compute run. This is the strongest model-quality outcome of the campaign: the remote setup did not beat the global tree best, but it did produce a better neural family anchor.

Program-Level Context

The campaign winner is not the global program leader.

According to `output/registries/program/current_best_solution.yaml`, the current program-level best solution remains:

- `te_hist_gbr_tabular`
- model type: `hist_gradient_boosting`
- test MAE: `0.002885 deg`
- test RMSE: `0.003607 deg`

The remote campaign winner `te_hist_gbr_remote_deep` reached:

- test MAE: `0.002920 deg`
- test RMSE: `0.003644 deg`

So the broader ranking after the remote pass is:

1. global program best still belongs to the earlier tree leader `te_hist_gbr_tabular`;
2. best remote-validation candidate is `te_hist_gbr_remote_deep`;
3. best feedforward family result is now `te_feedforward_high_compute_remote`.

Remote Workflow Validation Findings

The campaign was also a real infrastructure test, and that part produced high-value findings.

Fixed During The Validation Pass

Three launcher defects were exposed and corrected while running the real LAN campaign:

1. `System.IO.Path.GetRelativePath` was not available on the local PowerShell /.NET runtime.
2. Binary `tar | ssh` piping through PowerShell corrupted the stream on Windows.
3. Remote PowerShell progress records serialized as `CLIXML` and broke the local `ssh.exe` stream handling. The local repository launcher was hardened to handle these issues.

Remaining Workflow Anomaly

One bookkeeping inconsistency remains visible in the remote campaign artifacts:

- `te_random_forest_remote_medium` completed and wrote a real run directory `2026-04-03-20-38-08__te_random_forest_remote_medium`;
- the campaign manifest instead serialized `2026-04-03-20-38-07__te_random_forest_remote_medium`. This caused the automated sync manifest to point at a non-existent path. The run artifacts were recovered manually from the real remote directory. This is not a model failure. It is a remote bookkeeping bug in the current campaign-sync path and should be treated as follow-up workflow work before the launcher is promoted into a reusable repository skill.

Large-Artifact Warning

The medium random-forest run produced an extremely large serialized tree model:

- `tree_model.pkl` : `91,694,379,477 bytes`

  This is far above GitHub's regular object-size limit and confirms that the random-forest branch is operationally awkward even when the fit succeeds.

Main Conclusions

The remote validation campaign supports six concrete conclusions.

1. The LAN Remote Training Topology Is Real And Useful

The repository can now execute mixed tree and neural campaigns from the local terminal while using the stronger LAN workstation for the heavy runtime.

2. HistGradientBoosting Still Owns The Mainline Accuracy Lead

The deeper remote tree probe is good, but it still does not beat the earlier global best `hist_gradient_boosting` run.

3. Random Forest Is Still Not Worth Pushing As A Mainline Winner

The medium retry completed but did not improve the family meaningfully, and the aggressive retry still failed for memory reasons.

4. Remote GPU Training Improved The Feedforward Family Baseline

`te_feedforward_high_compute_remote` is now the best feedforward family result in the repository.

5. The Remote Launcher Is Close, But Not Yet Skill-Ready

It is strong enough for real use, but the remaining manifest/sync bookkeeping bug needs to be fixed before the workflow should be promoted into a reusable skill.

6. Artifact Size Must Be Treated As A First-Class Constraint

The successful medium random forest produced a model artifact on the order of `91 GB`, which is operationally heavy and GitHub-incompatible.

Recommended Next Actions

The next technically useful steps are

1. patch the remote campaign bookkeeping so the manifest always records the exact `run_instance_id` produced by the training entry point;
2. add a sync mode or recovery mode that can pull artifacts after a partially successful remote run without relaunching the whole campaign;
3. promote the remote-training workflow into a repository skill only after that recovery path is validated;
4. keep `te_feedforward_high_compute_remote` as the new feedforward reference point for future neural tuning;
5. keep the tree family anchored on `hist_gradient_boosting` rather than spending more effort on random forest.

Artifact References

Campaign-level references

- `output/training_campaigns/2026-04-03-20-38-07_remote_training_validation_campaign_2026_04_03_17_54_21/campaign_manifest.yaml`
- `output/training_campaigns/2026-04-03-20-38-07_remote_training_validation_campaign_2026_04_03_17_54_21/campaign_execution_report.md`
- `output/training_campaigns/2026-04-03-20-38-07_remote_training_validation_campaign_2026_04_03_17_54_21/campaign_leaderboard.yaml`
- `output/training_campaigns/2026-04-03-20-38-07_remote_training_validation_campaign_2026_04_03_17_54_21/campaign_best_run.yaml`
- `output/training_campaigns/2026-04-03-20-38-07_remote_training_validation_campaign_2026_04_03_17_54_21/campaign_best_run.md`

Key per-run references:

- `output/training_runs/tree/2026-04-03-21-48-11__te_hist_gbr_remote_deep/`
- `output/training_runs/feedforward/2026-04-03-21-50-07__te_feedforward_high_compute_remote/`
- `output/training_runs/feedforward/2026-04-03-22-00-31__te_feedforward_stride1_big_remote/`
- `output/training_runs/tree/2026-04-03-20-38-08__te_random_forest_remote_medium/`
- `output/training_runs/tree/2026-04-03-21-17-21__te_random_forest_remote_aggressive/`

Each completed run folder includes:

- `metrics_summary.yaml`
- `training_test_report.md`
- model artifact or checkpoint pointer